

Activity 4 - Practice Set React-Native

Integrating Payment Gateway:

- Select Payment Gateway:
 - Choose a payment gateway provider (e.g., Stripe, PayPal) that best suits your app's requirements and region.
- Set Up Payment Gateway Account:
 - Create an account with the chosen payment gateway provider and complete the necessary verification process.
- Obtain API Keys:
 - Retrieve API keys (e.g., publishable key, secret key) from the payment gateway provider's dashboard.
- Integrate Payment SDK:
 - Install and integrate the payment gateway's SDK into your mobile app project.
- Initialize Payment Gateway:
 - Initialise the payment gateway SDK with the API keys obtained earlier in your app's initialization process.
- Implement Payment Button:
 - Create a UI component (e.g., button) in your app's checkout or payment section to initiate the payment process.
- Handle Payment Events:
 - Implement logic to handle payment events such as success, failure, and cancellation.
- Tokenize Payment Information:
 - Ensure that sensitive payment information (e.g., credit card details) is securely tokenized before being sent to the payment gateway for processing.
- Test Payment Processing:
 - Use the payment gateway's sandbox environment to test payment processing and ensure that it works as expected.
- Handle Payment Responses:
 - Process payment responses received from the payment gateway to update the app's UI and provide feedback to the user.
- Implement Error Handling:
 - Handle any errors that may occur during the payment process and provide appropriate error messages to the user.
- Compliance and Security (Optional):

- Ensure that your app complies with payment industry standards (e.g., PCI DSS) and follows best practices for security and data protection.
- User Confirmation:
 - Provide users with a confirmation screen or message once the payment has been successfully processed.

or

1. The facilitator can decide what other options to explore for the API activity. The API activity should cover the HTTP Methods outlined in the Key Learning Content.

- API Activity Requirements:
 - Objective:
 - The API activity aims to familiarise trainees with integrating external APIs into their applications.
 - API Selection:
 - Trainees can choose any public API of their interest (e.g., weather, news, sports) for this activity.
 - Functionality:
 - Implement CRUD operations (Create, Read, Update, Delete) using the selected API.
 - Use appropriate API endpoints for each CRUD operation.
 - Authentication:
 - If the selected API requires authentication, trainees should implement the necessary authentication method (e.g., API keys, OAuth) in their application.
 - Data Handling:
 - Retrieve data from the API and display it in the application.
 - Implement logic to update or delete data through the API.
 - User Interface:
 - Design a user-friendly interface to interact with the API (e.g., forms for creating/updating data, lists for displaying data).
 - Error Handling:
 - Implement error handling mechanisms to manage API errors and provide meaningful error messages to the user.
 - Testing:
 - Test the API integration thoroughly to ensure that data is being retrieved, updated, and deleted correctly.
 - Use debugging tools (e.g., Postman) to troubleshoot any issues with the API integration.
 - Documentation:

- Provide documentation for the API integration, including details about the selected API, endpoints used, authentication method, and any other relevant information.
- Presentation:
 - Trainees should be prepared to present their API integration to the lab, explaining the API they chose, the functionality implemented, and any challenges faced during the integration process.
- Additional Options:
 - The facilitator may suggest additional requirements or options for the API activity based on the trainees' skill level and learning objectives.
 - Trainees can explore features like pagination, filtering, or sorting data from the API to enhance their understanding of API integration concepts.

External Resources

1. <https://rapidapi.com/blog/axios-react-api-tutorial/>
2. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch